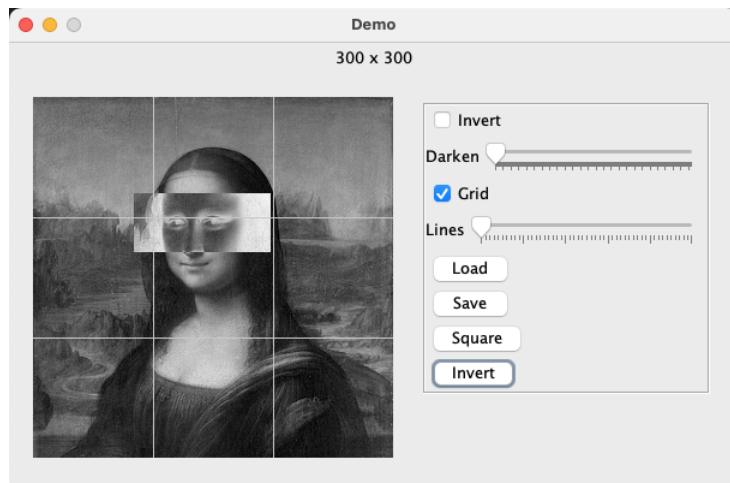




Vintage Editor

Introdução à Programação - ISCTE-IUL 2025/26 Projeto Individual

Neste projeto pretende-se desenvolver um editor de imagens a preto e branco (tons de cinzento), onde será possível aplicar diversos efeitos e operações nas mesmas.

Deverá ser usada a seguinte *framework* como base para o desenvolvimento:

<https://github.com/andre-santos-pt/greeditor>

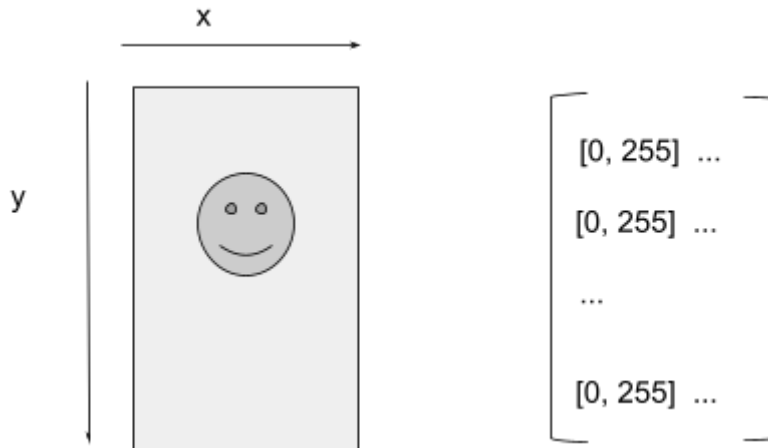
Plágio: O projeto é individual, e em nenhum caso deve ser copiado código que será entregue. Casos de plágio óbvio serão penalizados com a anulação da componente prática, e logo, implicam a reprovação à disciplina. Serão penalizados da mesma forma tanto os alunos que fornecem código como os que copiam código de outros.

Entrega: A submissão do projeto é feita exclusivamente no Moodle. Entregas com atraso poderão não ser aceites ou ter uma penalização. O prazo de entrega é na última aula da respetiva turma prática (semana de 9-12 de Dezembro).

Avaliação: Os estudantes deverão comparecer na sua turma prática para acompanhamento do projeto. A avaliação é feita presencialmente pelo professor da respetiva turma prática após submissão no Moodle. Poderá haver uma pequena sessão individual (5min) onde serão feitas perguntas sobre o código. Perante a incapacidade do estudante em responder às perguntas colocadas, o projeto pode ser desqualificado. A nota do projeto não dirá respeito apenas às funcionalidades implementadas, mas também se foi demonstrada uma boa compreensão do código desenvolvido.

Imagens

Uma imagem a preto e branco será representada numa matriz de inteiros, cujos valores devem estar no intervalo $[0, 255]$. Estes valores representam a luminosidade de cada píxel, correspondendo o zero a preto e 255 a branco. Qualquer valor fora deste intervalo é considerado inválido.

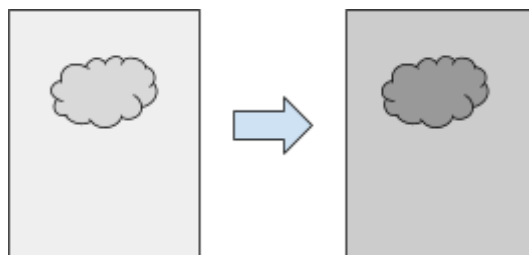


Por exemplo, a seguinte matriz em Java corresponde a uma imagem toda a preto, com 100 píxeis de altura e 50 de largura.

```
int[][] image = new int[100][50];
```

Filtros

Considere-se que um filtro consiste num efeito que converte o valor de cada píxel noutra valor, de forma independente, i.e. sem ter em conta a posição do píxel nem valores de outros píxeis. Por exemplo, para tornar uma imagem mais escura, podemos aplicar uma transformação que consiste em subtrair um dado valor a cada píxel.



A transformação de cada píxel é independente dos restantes, e logo, podemos representá-la como uma função $f(\text{inteiro}) \rightarrow \text{inteiro}$, onde o parâmetro corresponde ao valor original do píxel, e o retorno ao valor transformado. Por exemplo, para escurecer podemos ter a função:

```
int darken(int value) {  
    return value - 20;  
}
```

Atenção: a transformação apresentada não tem em consideração a possibilidade do valor retornado ser negativo.

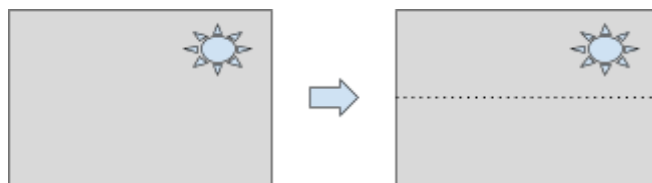
Ao aplicar esta função a cada píxel de uma imagem, teríamos como resultado uma imagem ligeiramente mais escura.

Um filtro pode ter um parâmetro inteiro. Por exemplo, para o exemplo podia ser passada a intensidade do escurecimento. Desta forma, temos a seguinte função, que permite variar a intensidade da aplicação do filtro.

```
int darken(int value, int intensity) {  
    return value - intensity;  
}
```

Efeitos

Um efeito, por oposição a um filtro, consiste numa transformação que o cálculo da transformação dos píxeis pode requer informação da posição (x, y) do mesmo, ou até de valores de outros píxeis. Por exemplo, se o efeito consiste em desenhar por cima da imagem a seguinte linha, teremos que ter em conta quais as posições que vão ser alteradas. Ou seja, apenas algumas posições sofrerão alterações (relativas à linha), mantendo-se os restantes valores inalterados.



Iremos representar efeitos através de um procedimento que altera uma matriz de inteiros. Se no exemplo o objetivo fosse desenhar uma linha a preto, teríamos o seguinte procedimento, onde é aplicada uma alteração aos píxeis da imagem.

```
void line(int[][] image) {  
    for(int x = 0; x < image[0].length; x++)  
        image[image.length/2][x] = 0;  
}
```

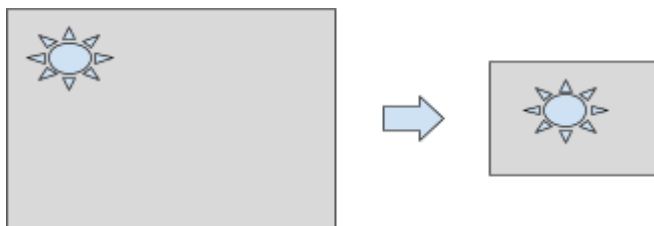
Operações

Uma operação consiste numa transformação que poderá resultar numa nova imagem, possivelmente com dimensões diferentes.

Iremos representar operações através de funções que devolvem (opcionalmente) uma imagem. Se for devolvida uma matriz, a imagem atual será substituída. Se for devolvido null, a imagem atual não é substituída.

No seguinte exemplo, temos a operação de reduzir a imagem à sua quarta parte superior esquerda.

```
int[][] topLeftCorner(int[][] image) {  
    int[][] corner = new int[image.length/2][image[0].length/2];  
    for(int y = 0; y < image.length/2; y++)  
        for(int x = 0; x < image[y].length/2; x++)  
            corner[y][x] = image[y][x];  
    return corner;  
}
```



Preparação

Seguir o tutorial do Greyditor e experimentar os exemplos fornecidos:

<https://github.com/andre-santos-pt/greyditor/blob/main/README.md>

Objetivos

Recomenda-se que o desenvolvimento do projeto decorra por fases, as quais devem corresponder às seguintes secções deste enunciado. Não será apropriado avançar para uma etapa mais avançada sem ter a etapa anterior minimamente dominada.

Fase 1

Pretende-se que sejam desenvolvidos os seguintes filtros:

1. **Clarear / escurecer** (filtro*)

Típico em programas de edição de imagem.

2. **Contraste** (filtro*)

Os píxeis mais claros ficam mais claros, ao passo que os mais escuros mais escuros.

3. **Espelhar horizontalmente** (efeito)

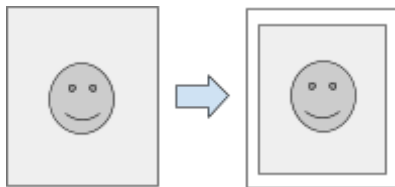
4. **Espelhar verticalmente** (efeito)

5. **Grão** (filtro*)

Introduzir ruído na imagem, por forma a causar “grão”. Com alguma aleatoriedade, aumentar ou reduzir a luminosidade dos píxeis.

6. **Margem** (efeito)

Sobrepor na imagem uma margem branca com um número *fixo* de píxeis de largura.



7. **Vignette** (efeito*)

Clarear/escurecer os cantos da imagem, regulando a intensidade.



* - deverá ter um parâmetro na interface gráfica (ao passo que na função/procedimento em si terá dois parâmetros)

Fase 2

Pretende-se que sejam desenvolvidas as seguintes operações:

1. **Cortar**

Cortar a imagem com base na seleção. (*crop*)

2. **Expandir**

Dados valores para altura e largura, produzir uma imagem com a nova dimensão, copiando os píxeis existentes e deixando a preto os “novos” píxeis.

3. **Posterizar**

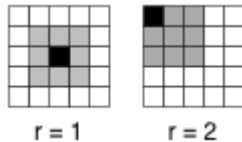
Produzir uma nova imagem “posterizada” dando o número de cores ([saber mais](#)).

4. Rodar

Rodar a imagem em 90 graus para a direita.

5. Desfocar (blur)

O efeito *blur* consiste em “esbater” os píxeis da imagem. Existem vários algoritmos para concretizar este efeito. Neste projeto pretende-se uma versão simples do mesmo que se explica de seguida.



A imagem alterada terá que ser construída de raiz, i.e. o efeito não poderá ser aplicado diretamente na imagem. O efeito deverá ter um parâmetro r que define a abrangência da vizinhança de cada píxel.

1. Criar uma imagem com as dimensões da original
2. O tom de cada píxel da imagem nova é calculado com base nos píxeis vizinhos da imagem original (num raio r), sendo uma média entre os valores dos píxeis vizinhos.

6. Antigo

Aplicar uma transformação que resulta da combinação de grão, vignette, e margem.

7. Retro

Aplicar uma transformação que resulta da combinação de contraste, desfoque (blur) e vignette.

Fase 3

Pretende-se que sejam desenvolvidas as seguintes operações que colaboram entre si:

1. Copiar

Copiar a área selecionada (a ficar guardada em memória como uma imagem).

2. Cortar (cut)

Semelhante ao copiar, mas a área copiada é apagada a branco.

3. Colar (paste)

Colar a imagem previamente guardada resultante da operação de copiar ou cortar (caso exista). Se a imagem guardada é resultante de copiar, pode ser colada diversas vezes. Caso a imagem guardada seja resultante de cortar, ao colar a mesma deixa de estar disponível para colar novamente.

4. Desfazer operação (undo)

Permitir desfazer as operações executadas.

Versão simples: apenas permite desfazer a última operação.

Versão elaborada: permite desfazer várias operações.